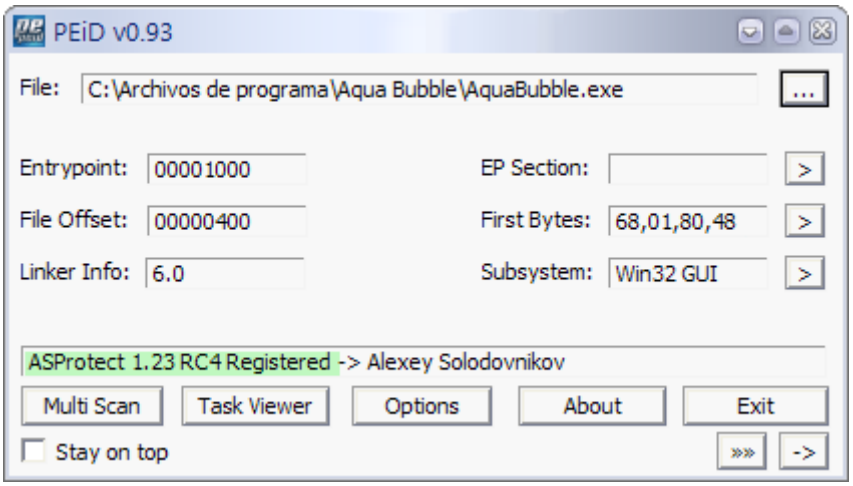
Estudio de ASProtect v1.23 RC4 Registered

Programa:	Aqua Bubble
Protección:	ASProtect v1.23 RC4 Registered + limitaciones varias
Objetivos:	Desempakar y lograr que funcione
URL descarga:	http://arcade.reflexive.com
Dificultad:	Sencilla
Herramientas:	OllyDbg v1.10 + PEiD v0.93 + ImpREC v1.6 + ConTEXT v0.98 + Hex WorkShop 4.23 + PEditor v1.7 + LordPE Deluxe
Cracker:	SyXe'05 .::[hAcK-c0d3d]..[CracksLatinoS]::. Fecha: 01-08-2006

Un saludo a todo CracksLatinoS!

Aquí estamos una vez más para tratar de resolver una protección comercial de las más conocidas como es ASProtect, intentaremos ver en este tutorial las diferencias que pueda haber con respecto a la versión 1.23 RC4 normal [sin registrar] si es que las hubiera.

Lo primero que haremos será instalar la app en nuestro querido PC y echarle un vistazo a su ejecutable con PEiD v0.93 a ver qué se cuenta:



En este caso ya lo sabíamos pero no siempre va a ser así, es conveniente acostumbrarse a mirar siempre el EXE con PEiD, una vez que sabemos de qué protección se trata podemos comenzar.

Echando un primer vistazo..

Pues eso mismo, cómo ya sabemos la manera en que trabaja este packer no ahondaremos demasiado en ello. Comenzaremos por cargar el ejecutable "AquaBubble.exe" en Ollydbg.

Antes de continuar he de decir que tengo instalado en él los siguientes plugins:

1.- CommandBar v3.10.109c de Oleg Yuschuk
2.- Hide Debugger v1.2.3f de Asterix
3.- IsDebuggerPresent v1.4 (SV 2003)
4.- OllyDump v2.21.108 de Gigapede
5.- OllyScript v0.92 de SHaG

Supongo que serán bien conocidos por todos nosotros, el segundo y tercero sirven prácticamente para lo mismo aunque el Hide Debugger trae algunas opciones más que no trae el IsDebuggerPresent, de todas maneras sabemos que este packer únicamente se protege mediante el byte IsDebuggerPresent que en el EP encontramos en [ebx+2] en todas las máquinas [7FFDF002] y cuyo byte bajo debe estar a 0 para no ser detectados.

Dicho esto seguimos. Los pasos para desempakar un ASProtect creo que están más que aclarados puesto que se han escrito cientos de tutes acerca de él, por esto mismo lo vamos a hacer rapidito. Este es su EntryPoint:

```
Address   Hex dump        Disassembly                          Comment
00401000  68 01804800     push    00488001
00401005  E8 01000000     call    0040100B
0040100A  C3              ret
0040100B  C3              ret
0040100C  8391 B8E1508E   adc     dword ptr [ecx+8E50E1B8],-5B
00401013  A0 1781DC7D     mov     al,[7DDC8117]
00401018  6D              insd                                 I/O command
00401019  42              inc     edx
0040101A  B1 A6           mov     cl,0A6
0040101C  2E:07           pop     es                           Modification of segm
0040101E  9A 1F7D7F1D 95  call    far 9295:1D7F7D1F            Far call
00401025  B5 A4           mov     ch,0A4
00401027  7A 34           jpe     short 0040105D
00401029  19A0 E8180361   sbb     [eax+610318E8],esp
```

Típica estructura de estas versiones de ASProtect, lo siguiente es llegar al OEP con el método de Ricardo de las excepciones, la última es ésta:

```
Address   Hex dump        Disassembly                          Comment
01283D03  3100            xor     [eax],eax
01283D05  64:8F05 000000  pop     dword ptr fs:[0]
01283D0C  58              pop     eax
01283D0D  833D BC7E2801   cmp     dword ptr [1287EBC],0
01283D14  74 14           je      short 01283D2A
```

Ponemos un BPM en la sección de código y la pasamos con SHIFT+F9 y llegamos a lo que sería el OEP aunque en realidad no es este realmente, veámoslo:

```
Address   Hex dump        Disassembly                          Comment
00437431  0000            add     [eax],al
00437433  0000            add     [eax],al
00437435  0000            add     [eax],al
00437437  0000            add     [eax],al
00437439  0000            add     [eax],al      ---> Stolen bytes!
0043743B  0000            add     [eax],al
0043743D  0000            add     [eax],al
0043743F  0000            add     [eax],al
00437441  0000            add     [eax],al
00437443  0000            add     [eax],al
00437445  FF15 68914500   call    [459168]
0043744B  33D2            xor     edx,edx
0043744D  8AD4            mov     dl,ah
0043744F  8915 E8064700   mov     [4706E8],edx
```

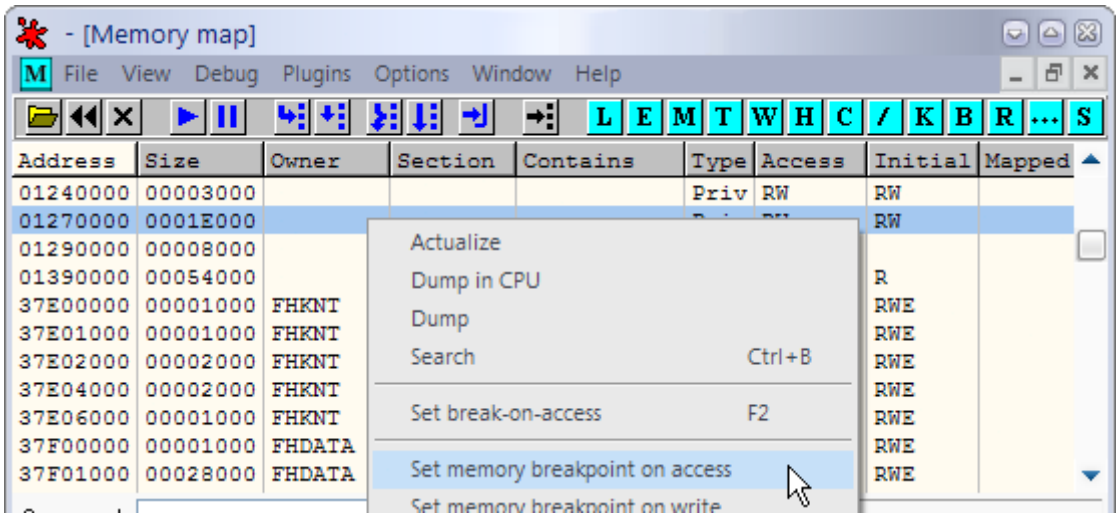
como se ve en la imagen falta código antes del call donde Olly paró, son los famosos stolen bytes (bytes robados) que a continuación buscaremos, la manera de encontrarlos pueden ser varias, aquí veremos sólo una de ellas…

Buscando los stolen bytes..

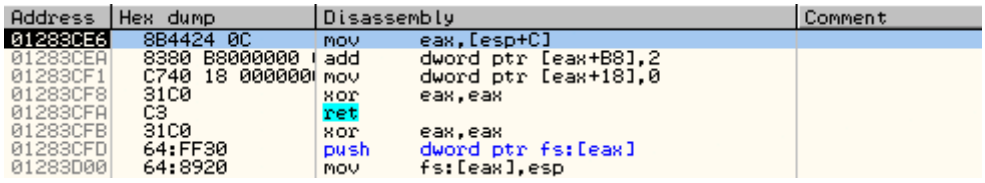
Reiniciamos Olly con CTRL+F2 y llegamos hasta la última excepción nuevamente, miramos la pila y buscamos el SEH (manejador de excepciones):

```
Address   Value     Comment
0012FF64  01290BB0  Pointer to next SEH record
0012FF68  01283CE6  SE handler <---
0012FF6C  0012FF90
0012FF70  00000003
0012FF74  01250000
0012FF78  01290BC4
```

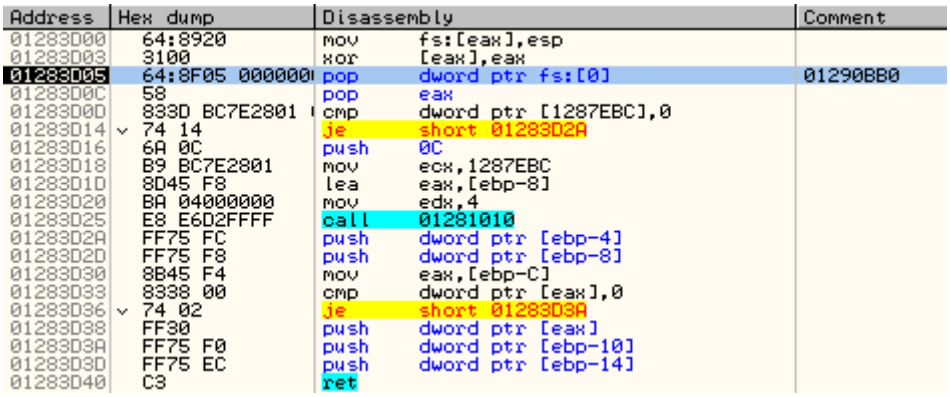
Vemos que la excepción se manejará a partir de la dirección :01283CE6 que forma parte de la sección de memoria donde está trabajando ahora mismo el packer, pues bien, ponemos un BPM ON MEMORY - ACCESS que abarque toda ella:



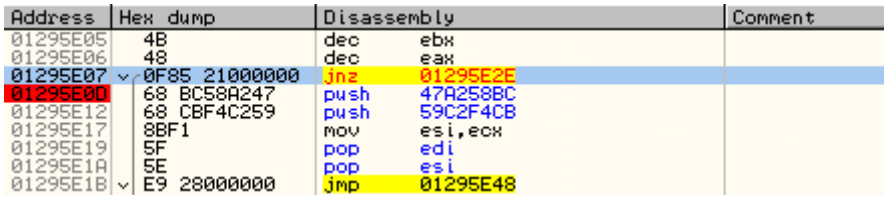
Pulsamos CTRL+F9 para pasar la excepción y paramos en esta parte:



Traceamos hasta el RET que nos deja en NTDLL.DLL pulsamos F9 ya que tenemos el BPM todavía puesto y volvemos a parar en la misma sección un poco después del código anterior:



Pasamos la zona con F8 hasta el RET que nos deja en esta misma sección esta vez, a partir de ahora traceamos con F7 con cuidado, veremos algunos loops que iremos deshaciendo colocando un BP justo después como en este caso:



Y continuamos con F7 igual entrando en los call que can apareciendo, pues de no hacerlo así el programa cargaría sin poder hacer nada para impedirlo, llegará un momento (no muy tarde) donde hará acto de presencia otro buclecillo también pequeño donde hay un call, es este:

```
Address   Hex dump     Disassembly                      Comment
01295E71  0FB703       movzx   eax,word ptr [ebx]
01295E74  03C6         add     eax,esi
01295E76  83C3 02      add     ebx,2
01295E79  8BD0         mov     edx,eax
01295E7B  8BC6         mov     eax,esi
01295E7D  E8 0C000000  call    01295E8E
01295E82  66:833B 00   cmp     word ptr [ebx],0
01295E86  75 E9        jnz     short 01295E71
01295E88  5E           pop     esi
01295E89  5B           pop     ebx
01295E8A  5D           pop     ebp
01295E8B  C2 0800      ret     8
```

Lo deshacemos de la misma forma poniendo un BP después del JNZ y pulsando F9, seguimos traceando con cuidado porque cuando pasamos el RET 8 entramos en la zona donde ejecuta los stolen bytes, éste es su inicio:

```
Address   Hex dump     Disassembly                      Comment
01295EEC  65:EB 01     jmp     short 01295EF0           Superfluous prefix
01295EEF  F2:          prefix repne:                    Superfluous prefix
01295EF0  F2:          prefix repne:                    Superfluous prefix
01295EF1  EB 02        jmp     short 01295EF5
01295EF3  CD 20        int     20
01295EF5  FF7424 13    push    dword ptr [esp+13]
01295EF9  FF7424 07    push    dword ptr [esp+7]
01295EFD  896C24 04    mov     [esp+4],ebp
01295F01  F2:          prefix repne:                    Superfluous prefix
01295F02  EB 01        jmp     short 01295F05
01295F04  F2:          prefix repne:                    Superfluous prefix
01295F05  83EC 1B      sub     esp,1B
01295F08  83C4 1F      add     esp,1F
01295F0B  EB 02        jmp     short 01295F0F
01295F0D  CD 20        int     20
01295F0F  8B6C24 10    mov     ebp,[esp+10]
01295F13  EB 02        jmp     short 01295F17
01295F15  CD 20        int     20
01295F17  2BE8         sub     ebp,eax
```

Se puede ver que es totalmente distinta a la anterior y fácil de diferenciar, a partir de aquí vamos con mucho cuidado atento a los registros y la pila porque como digo se van a ejecutar los bytes que buscamos pero entremezclados con código basura para despistar, además esta zona es automodificable ya que se va desencriptando mediante algunos XOR en medio de la rutina, traceamos con F7 hasta llegar aquí:

```
Address   Hex dump     Disassembly                      Comment
01295F8A  FF7424 1B    push    dword ptr [esp+1B]
01295F8E  FF7424 13    push    dword ptr [esp+13]
01295F92  896C24 04    mov     [esp+4],ebp  <-- inicio
01295F96  3E:EB 02     jmp     short 01295F9B           Superfluous prefix
01295F99  CD 20        int     20
01295F9B  83EC 1C      sub     esp,1C
01295F9E  83C4 20      add     esp,20
01295FA1  8BEC         mov     ebp,esp
01295FA3  6A FF        push    -1
01295FA5  68 88AF4500  push    45AF88
01295FAA  68 9CA14300  push    43A19C
01295FAF  64:A1 00000000 mov   eax,fs:[0]
01295FB5  EB 01        jmp     short 01295FB8
01295FB7  E8 668105C2  call    C32EE122
01295FBC  5F           pop     edi
01295FBD  2901         sub     [ecx],eax
01295FBF  DCC8         fmul    st,st
01295FC1  F2:          prefix repne:                    Superfluous prefix
01295FC2  0F39         ???                              Unknown command
01295FC4  9A FF74241B FF call  far 74FF:1B2474FF        Far call
01295FCB  24 13        and     al,13
01295FCD  894424 04    mov     [esp+4],eax
01295FD1  3E:EB 02     jmp     short 01295FD6           Superfluous prefix
01295FD4  CD 20        int     20
01295FD6  83EC 1C      sub     esp,1C
01295FD9  83C4 20      add     esp,20
01295FDC  64:8925 000000 mov   fs:[0],esp
01295FE3  83EC 58      sub     esp,58
01295FE6  EB 01        jmp     short 01295FE9
01295FE8  E8 668105F3  call    F42EE153
01295FED  5F           pop     edi
```

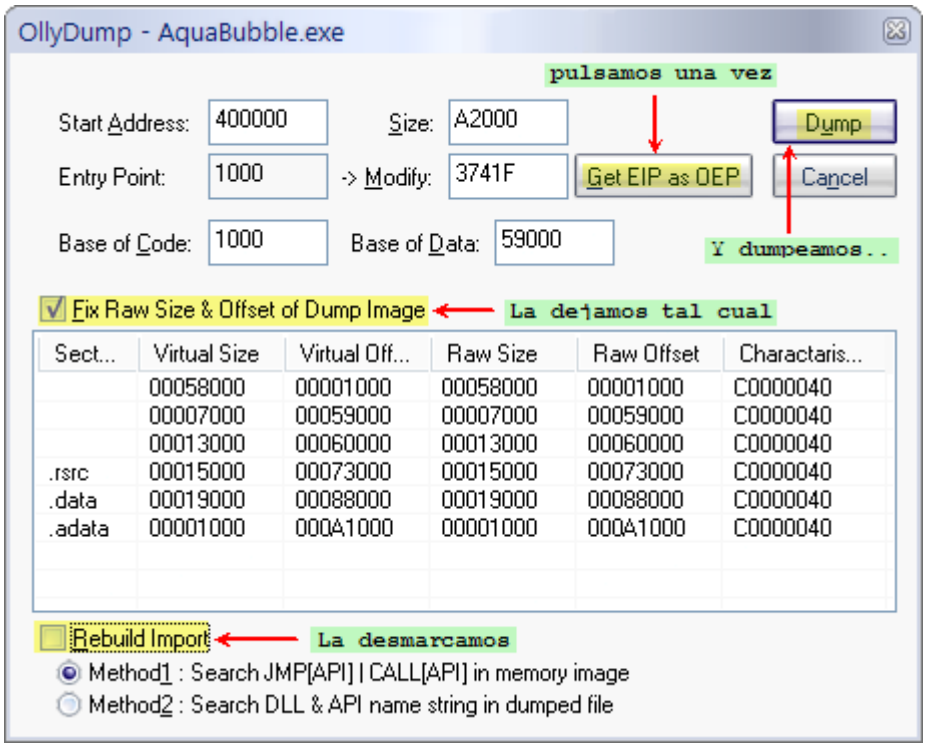
Donde comienzan a vislumbrarse los stolen bytes que empiezan en :01295F92 con un "PUSH EBP" camuflado, si seguimos traceando los vamos encontrando todos:

```
01295FFE    895C24 04     mov     [esp+4],ebx          ; PUSH EBX
01296002    3E:EB 02      jmp     short 01296007       ; Superfluous prefix
01296025    897424 04     mov     [esp+4],esi          ; PUSH ESI
01296029    3E:EB 02      jmp     short 0129602E       ; Superfluous prefix
0129604C    897C24 04     mov     [esp+4],edi          ; PUSH EDI
01296050    3E:EB 02      jmp     short 01296055       ; Superfluous prefix
0129605B    8965 E8       mov     [ebp-18],esp         ; MOV [EBP-18],ESP
0129605E    EB 01         jmp     short 01296061
```

Los colocamos en el "verdadero" OEP [por encima del call] quedando así:

```
Address   Hex dump        Disassembly                    Comment
0043741F  55              push   ebp
00437420  8BEC            mov    ebp,esp
00437422  6A FF           push   -1
00437424  68 88AF4500     push   0045AF88
00437429  68 9CA14300     push   0043A19C
0043742E  64:A1 00000000  mov    eax,fs:[0]
00437434  50              push   eax
00437435  64:8925 000000  mov    fs:[0],esp
0043743C  83EC 58         sub    esp,58
0043743F  53              push   ebx
00437440  56              push   esi
00437441  57              push   edi
00437442  8965 E8         mov    [ebp-18],esp
00437445  FF15 68914500   call   [459168]
0043744B  33D2            xor    edx,edx
```

Hacemos un NEW ORIGIN HERE al inicio de éstos antes de dumpear con el propio Ollydbg y las opciones de siempre:



Al dumpeado lo llamaremos "tute.exe" por ejemplo pero todavía no funcionará porque falta por reparar la IAT que repararemos a continuación.

Reparando la IAT..

Los pasos para reparar la tabla en estas versiones están también muy explicados y bien detallados en otros tutes anteriores así que no nos detendremos demasiado en el tema. Daremos un breve repaso nada más.

Ya podemos cerrar el Olly donde tenemos el programa parado en el OEP pues no nos hará más falta, no sin antes fijarnos en el inicio y final de la IAT:

```
Address   Hex dump                                         ASCII
00459000  A0 7D DA 77 10 24 DA 77 BB 28 DA 77 B1 63 DB 77   }Úw+$Úw»(Úw±cÛw
00459010  D9 23 DA 77 AA A9 DA 77 90 AC DA 77 4F A5 DA 77  Ù#Úwª©Úw¬ÚwO¥Úw
00459020  CF AA DA 77 06 A7 DA 77 6B 82 DA 77 DE A5 DA 77  ÏªÚw§Úwk‚Úw¥Úw
00459030  D8 17 DA 77 00 00 00 00 CB 52 AD 73 50 E2 AD 73  Ø↨Úw....ER-sPâ-s
00459040  63 E2 AD 73 41 56 AD 73 FF E2 AD 73 68 54 AD 73  câ-sAV-sÿâ-shT-s
00459050  2D 57 AD 73 10 56 AD 73 46 58 AD 73 CB 52 AD 73  -W-s►V-sFX-sER-s
00459060  C3 55 AD 73 16 54 AD 73 67 59 AD 73 C8 53 AD 73  ÃU-s▬T-sgY-sES-s
00459070  63 52 AD 73 00 00 00 00 B1 38 31 77 00 00 00 00  cR-s....±81w....
00459080  FD CD 01 51 00 00 00 00 CC 16 0A 51 00 00 00 00  ýÍ☺Q....Ì▬◙Q....
```

Este es su inicio que luego anotaremos, antes busquemos su final:

Address	Hex dump	ASCII
00459420	30 AA D1 77 15 8A D1 77 C7 8F D1 77 67 88 D1 77	0ªÑw←ŠÑwÇ□Ñwg^Ñw
00459430	85 D1 D1 77 3F E9 D1 77 5E 8B D2 77 00 00 00 00	_ÑÑw?éÑw^‹Ów....
00459440	5C 46 B1 76 F2 4F B1 76 5E 93 B0 76 CC 9B B0 76	\F±vòO±v^"°vÌ›°v
00459450	02 04 B1 76 D9 14 B1 76 F4 12 B1 76 2C 92 B0 76	♫♦±vÙ¶±vô↕±v,'°v
00459460	69 95 B0 76 2C 97 B0 76 32 63 B1 76 7E 91 B0 76	i•°v,—°v2c±v~'°v
00459470	00 00 00 00 AD 5F F8 72 69 5E F8 72 63 16 F8 72	....¯_ørî^ørc▬ør
00459480	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00459490	D7 A3 F0 3E F6 28 BC 3F C0 11 40 00 90 12 40 00	x£ð>ö(¼?À◄@.►↕@.
004594A0	00 00 F0 43 D0 0F 49 3C F0 13 40 00 D0 15 40 00	..ðCÐ☼I<ð‼@.Ð§@.

La última entrada válida está en [45947C]==72F81663, los datos de tabla serán:

inicio: 459000
final: 459480
largo: 480

Ahora sí cerramos este Olly y abrimos otro donde cargaremos el ejecutable original empakado, ponemos un BPM ON MEMORY - WRITE en la primera entrada [459000], antes deshabilitamos las excepciones para que Olly no pare en ellas, pulsamos ALT+O y marcamos todas las casillas (INT3 breaks, Single-step, etc..) pulsamos F9 y para en un REP MOVS, no vale, F9 otra vez y estamos aquí:

Address	Hex dump	Disassembly	Comment
01283641	56	push esi	
01283642	8B45 0C	mov eax,[ebp+C]	
01283645	50	push eax	
01283646	E8 C9FBFFFF	call 01283214	
0128364B	8B17	mov edx,[edi]	
0128364D	8902	mov [edx],eax	ADVAPI32.CryptAcquireContextA
0128364F	EB 09	jmp short 0128365A	

Si miramos por encima vemos el call mágico:

Address	Hex dump	Disassembly	Comment
012835C7	56	push esi	
012835C8	8B45 0C	mov eax,[ebp+C]	
012835CB	50	push eax	
012835CC	E8 43FCFFFF	call 01283214	
012835D1	E8 7EFEFFFF	call 01283454	// call magico
012835D6	8B17	mov edx,[edi]	
012835D8	8902	mov [edx],eax	
012835DA	EB 7E	jmp short 0128365A	

Lo que hacemos ahora es reiniciar Olly, desmarcamos las excepciones para que Olly pare nuevamente en ellas y pulsamos F9, cuando pare en la primera ponemos un BP en el call mágico, volvemos a marcar las excepciones para que no molesten por ahora, pasamos la excepción con SHIFT+F9 y caemos en el mágico que NOPeamos:

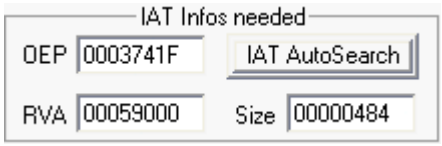
Address	Hex dump	Disassembly	Comment
012835C7	56	push esi	
012835C8	8B45 0C	mov eax,[ebp+C]	
012835CB	50	push eax	
012835CC	E8 43FCFFFF	call 01283214	
012835D1	90	nop	
012835D2	90	nop	
012835D3	90	nop	
012835D4	90	nop	
012835D5	90	nop	
012835D6	8B17	mov edx,[edi]	
012835D8	8902	mov [edx],eax	
012835DA	EB 7E	jmp short 0128365A	

Pulsamos CTRL+F9 para retornar de la función y ponemos un BP en el POPAD:

Address	Hex dump	Disassembly	Comment
01283864	5B	pop ebx	01296598
01283865	EB CE	jmp short 01283835	
01283867	61	popad	
01283868	E8 3A000000	call 012838A7	
0128386D	68 76382801	push 1283876	
01283872	FF0424	inc dword ptr [esp]	
01283875	C3	ret	

Esto lo hacemos para que se detenga cuando haya rellenado todas las entradas, también podríamos haber puesto un BPM en la última entrada pero como hay veces en que no termina por el final pues... que este método me gusta más, pulsamos F9 y al momento para en el POPAD, vamos para atrás con la tecla "-" y dejamos el call como estaba con la opción UNDO SELECTION para que no detecte ningún cambio ya que sino no correría muy probablemente. Volvemos a activar las excepciones (desmarcando las casillas) y llegamos al OEP con el mismo método de antes (el de Ricardo), abrimos el Import Reconstructor v1.6 y donde dice ATTACH TO AN ACTIVE

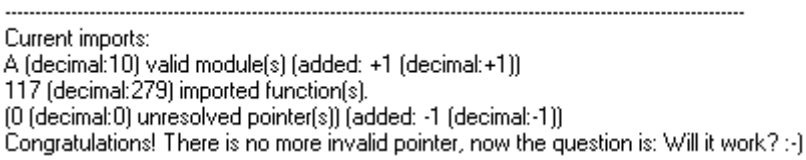
PROCESS seleccionamos el proceso AQUABUBBLE.EXE, le damos los valores correctos que serán estos para este programa:



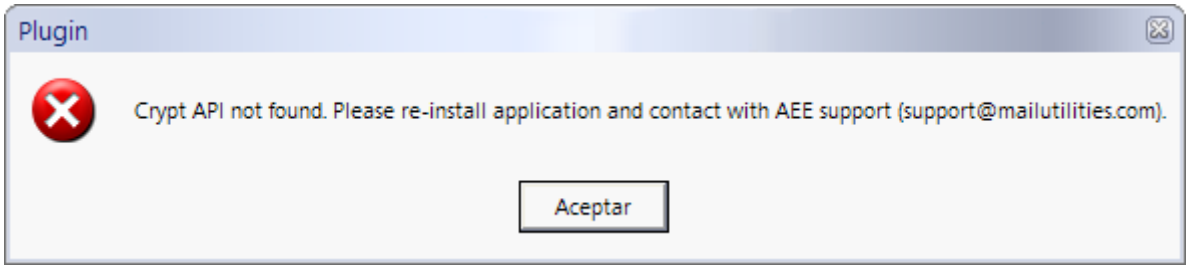
Pulsamos GET IMPORTS y nos dice que quedaron 6 entradas por reparar y que el mágico no pudo ayudarnos con ellas, por su puesto son muy fácil de arreglar con el mismo plugin del ImpRec, son éstas:

Entrada	Iat rva	apunta a	api correcta
1.-	59168	01281528	GetVersion
2.-	591B4	01281554	GetCurrentProcess
3.-	59220	01281574	GetCurrentProcess
4.-	59284	0128155C	GetCurrentProcessId
4.-	592A8	012810AC	GetProcAddress
5.-	592AC	01281500	GetModuleHandleA

Cuando terminamos de repararlas todas vemos esto en la ventana LOG:



Picamos en FIX DUMP y le damos tute.exe, nos dejará otro llamado tute_.exe al lado que contendrá la IAT reparada, lo ejecutamos y...



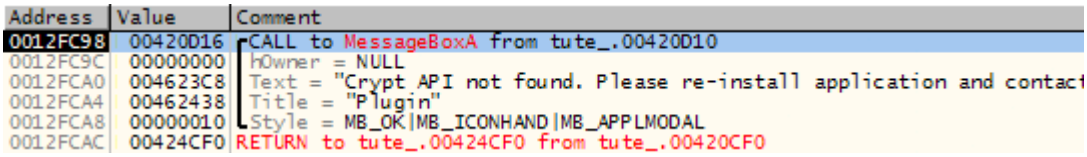
Vaya parece que no le gustaron los cambios, jeje.. investiguemos!

Comienza otra batalla..

Pues sí así es, muchas veces cuando desempakamos un ejecutable éste no funciona, en la mayoría de ocasiones esto es debido a algún antidump, en algunos casos propio del packer y en otro del propio programa, por el mensaje anterior parece ser que podría ser del packer.

Cargamos el dumpeado en el Olly y ponemos un BP en la api:

Damos RUN con F9 y se detiene en la api, vemos la pila:



La llamada se hizo en :420D10, vayamos hasta allí y echemos un vistazo:

```
Address   Hex dump     Disassembly                          Comment
00420CEF     90        nop
00420CF0  $  6A 04     push    4                            ProcNameOrOrdinal = #4
00420CF2  .  6A FF     push    -1                           hModule = FFFFFFFF
00420CF4  .  FF15 A892450  call  [<&kernel32.GetProcAddres  GetProcAddress
00420CFA  .  85C0      test    eax,eax
00420CFC  .  A3 30DE4600  mov   [46DE30],eax
00420D01  .v 75 16     jnz     short 00420D19
00420D03  .  6A 10     push    10                           Style = MB_OK|MB_ICONHAND|MB_
00420D05  .  68 38244600  push 00462438                     Title = "Plugin"
00420D0A  .  68 C8234600  push 004623C8                     Text = "Crypt API not found.
00420D0F  .  50        push    eax                          hOwner
00420D10  .  FF15 E093450  call  [<&user32.MessageBoxA>]    MessageBoxA
00420D16  .  33C0      xor     eax,eax
00420D18  .  C3        ret
00420D19  >  B8 01000000  mov  eax,1
00420D1E  .  C3        ret
```

Vemos una llamada a GetProcAddress previa al mensaje de error, lo que más extraña es ese "hModule = FFFFFFFF" normalmente se le pasan dos parámetros a esta api que son: Nombre de la api y imagebase de la DLL donde se encuentra ésta, pero este esquema no se cumple en este caso, para salir de dudas cargamos el EXE protegido en Olly y llegamos a esta zona, entramos en el call y vemos:

```
Address   Hex dump     Disassembly                          Comment
012810AC     55        push    ebp
012810AD     8BEC      mov     ebp,esp
012810AF     8B55 0C   mov     edx,[ebp+C]
012810B2     8B45 08   mov     eax,[ebp+8]
012810B5     8B0D 74642801  mov  ecx,[1286474]
012810BB     8B09      mov     ecx,[ecx]
012810BD     3BC8      cmp     ecx,eax
012810BF  v  75 09     jnz     short 012810CA
012810C1     8B0495 506328  mov  eax,[edx*4+1286350]      // recoge el valor
012810C8  v  EB 07     jmp     short 012810D1
012810CA     52        push    edx
012810CB     50        push    eax
012810CC     E8 E343FFFF  call  012754B4                   jmp to kernel32.GetProcAddress
012810D1     5D        pop     ebp
012810D2     C2 0800   ret     8
```

Se trata de GetProcAddress claramente pero en este caso no se ejecuta la api sino que recoge un valor en EAX en la dirección :012810C1 y luego salta por encima de la api retornando simplemente ese valor que lee de esta tabla:

```
Address   Hex dump                                         ASCII
01286300  44 BF 27 01 50 BF 27 01 6C C9 27 01 8C C9 27 01  D¿'☺P¿'☺l╔'☺Œ╔'☺
01286310  0F 00 00 00 FF FF FF FF 08 00 00 00 00 00 00 00  ☼...ÿÿÿÿ◘.......
01286320  00 00 00 00 74 15 28 01 28 15 28 01 34 15 28 01  ....t↨(☺(↨(☺4↨(☺
01286330  00 15 28 01 54 15 28 01 5C 15 28 01 64 15 28 01  .↨(☺T↨(☺\↨(☺d↨(☺
01286340  8C 15 28 01 9C 15 28 01 B0 15 28 01 F8 14 28 01  Œ↨(☺œ↨(☺°↨(☺ø¶(☺
01286350  00 00 00 00 00 00 00 00 00 00 00 00 5C 11 28 01  ............\◄(☺
01286360  04 12 28 01 F4 13 28 01 FC 13 28 01 0C 14 28 01  ♦↕(☺ô‼(☺ü‼(☺.¶(☺
01286370  20 14 28 01 34 14 28 01 08 11 28 01 30 11 28 01   ¶(☺4¶(☺◘◄(☺0◄(☺
01286380  0D 0A 20 09 27 22 8B C0 48 C9 27 01 02 00 00 00  ♪◙ ○'"‹ÀHÉ'☺...
```

Dejando EAX==01281204 que es el valor resaltado, no sé si es porque estamos en la versión Registrada pero nunca antes vi este comportamiento en esta api, siempre había llegado a ejecutarse ésta y no es el caso, de todas formas ahora veremos que este truco es muy fácil de pasar, veamos donde apunta EAX:

```
Address   Hex dump     Disassembly                          Comment
01281204     55        push    ebp
01281205     8BEC      mov     ebp,esp
01281207     53        push    ebx
01281208     56        push    esi
01281209     8B75 0C   mov     esi,[ebp+C]
0128120C     8B5D 08   mov     ebx,[ebp+8]
0128120F     56        push    esi
01281210     8BCB      mov     ecx,ebx
01281212     8B15 FC7C2801  mov  edx,[1287CFC]
01281218     A1 F87C2801  mov  eax,[1287CF8]
0128121D     E8 EEFDFFFF  call  01281010
01281222     56        push    esi
01281223     53        push    ebx
01281224     E8 07000000  call  01281230
01281229     5E        pop     esi
0128122A     5B        pop     ebx
0128122B     5D        pop     ebp
0128122C     C2 0800   ret     8
0128122F     90        nop
```

Es una pequeña rutina que vete tú a saber qué hace ¿?

Como comento es una pequeña rutina a la que nunca salta, lo podemos comprobar colocando un BP con F2 en el "PUSH EBP" inicial y pulsando F9, el programa arrancará sin parar en el BP. En el dumpeado haremos una pequeña modificación entonces para que en lugar de saltar a GetProcAddress ponga un valor distinto de cero en EAX y listos:

```
Address   Hex dump     Disassembly                      Comment
00420CF0  $ 90         nop
00420CF1  . 90         nop
00420CF2  . 90         nop
00420CF3  . 90         nop
00420CF4  . 90         nop
00420CF5  . B8 01000000 mov     eax,1
00420CFA  . 85C0       test    eax,eax
00420CFC  . A3 30DE4600 mov     [46DE30],eax
00420D01  .v 75 16     jnz     short 00420D19
00420D03  . 6A 10      push    10                       Style = MB_OK|MB_ICONHAND|MB_APPLMODA
00420D05  . 68 38244600 push   00462438                 Title = "Plugin"
00420D0A  . 68 C8234600 push   004623C8                 Text = "Crypt API not found. Please r
00420D0F  . 50         push    eax                      hOwner => 00000001
00420D10  . FF15 E093450 call  [<&user32.Message        MessageBoxA
00420D16  . 33C0       xor     eax,eax
00420D18  . C3         ret
00420D19  > B8 01000000 mov    eax,1
00420D1E  . C3         ret
```

Salvamos los cambios y probamos el nuevo EXE a ver qué tal, hacemos doble clic sobre él y vuelve a dar un petardazo, esta vez un ACCESS_VIOLATION aquí:

```
Address   Hex dump     Disassembly                      Comment
00424B9C  . 6A 0C      push    0C
00424B9E  . E8 ED060000 call   00425290
00424BA3  . A1 B0234600 mov    eax,[4623B0]             Aquí recoge el valor!
00424BA8  . 8B5424 10  mov     edx,[esp+10]
00424BAC  . 8B7424 14  mov     esi,[esp+14]
00424BB0  . 83C4 0C    add     esp,0C
00424BB3  . 8B08       mov     ecx,[eax]                Y falla!!
00424BB5  . 33CA       xor     ecx,edx
00424BB7  . 8B50 04    mov     edx,[eax+4]
00424BBA  . 8B40 08    mov     eax,[eax+8]
00424BBD  . 33D6       xor     edx,esi
```

Si miramos lo que hay en [4623B0] vemos que allí está el problema:

```
Address   Hex dump                                         ASCII
004623B0  B0 0B 29 01 0B 00 00 00 39 6D 46 55 65 41 41 41  °♂)☺♂...9mFUeAAA
004623C0  49 31 65 3D 00 00 00 00 43 72 79 70 74 20 41 50  I1e=....Crypt AP
004623D0  49 20 6E 6F 74 20 66 6F 75 6E 64 2E 20 50 6C 65  I not found. Ple
004623E0  61 73 65 20 72 65 2D 69 6E 73 74 61 6C 6C 20 61  ase re-install a
004623F0  70 70 6C 69 63 61 74 69 6F 6E 20 61 6E 64 20 63  pplication and c
00462400  6F 6E 74 61 63 74 20 77 69 74 68 20 41 45 45 20  ontact with AEE
```

Es evidente que en nuestro dumpeado no existe la posición [01290BB0] y por eso falla, no pasa esto en el original donde hay esto:

```
Address   Hex dump                                         ASCII
01290BB0  33 52 70 74 4C 77 42 67 61 4A 4D 3D 00 00 00 00  3RptLwBgaJM=....
01290BC0  70 74 28 01 70 74 28 01 20 3C 00 00 00 00 00 00  pt(☺pt(☺ <......
01290BD0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
01290BE0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
01290BF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
01290C00  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Esta cadena ya aparece en el falso OEP donde EAX tiene un puntero a ella, no estoy del todo seguro [no lo he verificado] pero creo que tiene mucho que ver con el tema del estado del programa [Trial/Registered/Expired] de todos modos también es fácil saltarse este truco, simplemente hacemos este cambio:

```
Address   Hex dump                                         ASCII
004623B0  B8 23 46 00 0B 00 00 00 39 6D 46 55 65 41 41 41  ¸#F.♂...9mFUeAAA
004623C0  49 31 65 3D 00 00 00 00 43 72 79 70 74 20 41 50  I1e=....Crypt AP
004623D0  49 20 6E 6F 74 20 66 6F 75 6E 64 2E 20 50 6C 65  I not found. Ple
004623E0  61 73 65 20 72 65 2D 69 6E 73 74 61 6C 6C 20 61  ase re-install a
004623F0  70 70 6C 69 63 61 74 69 6F 6E 20 61 6E 64 20 63  pplication and c
00462400  6F 6E 74 61 63 74 20 77 69 74 68 20 41 45 45 20  ontact with AEE
```

Lo único que hemos hecho es desviar la atención si se puede decir así hacia la cadena que tenemos justo después "9mFUeAAAI1e=" que nos viene como anillo al dedo ya que nos evita tener que escribir nosotros mismos una cualquiera, con esto conseguimos evitar el error que se generaba antes al intentar leer de una posición inexistente, seleccionamos los 4 bytes modificados y hacemos COPY TO EXECUTABLE FILE - SAVE FILE, lo salvamos con el mismo nombre y tendremos un archivo con extensión .bak que será la copia de seguridad por si las moscas.

Si probamos el ejecutable con el cambio anterior [más el de antes] comprobamos que funciona a la perfección. Lo probamos "a fondo" y aparentemente funciona. Concluimos entonces que está desempakado. Vamos a por las limitaciones xD

Registrando el programa ;)

A eso vamos, nada más ejecutarlo nos topamos con una especie de NagScreen que parece una especie de banner dado que no tiene botones de Cerrar, Maximizar, Minimizar, etc., es un fondo con tres botones en la parte baja, es ésta:

Nos invita a registrarnos además de informar de los beneficios de hacerlo, que no sufra que en cinco o diez minutos si todo va bien lo estaremos, jeje..

Nosotros atacaremos por la variable cadena que vimos antes, en realidad ésta nos servirá como un principio, después veremos.. esta es la zona:

```
Address   Hex dump          Disassembly                Comment
00424BA3  .   A1 B0234600   mov     eax,[4623B0]       // puntero a la cadena
00424BA8  .   8B5424 10     mov     edx,[esp+10]       // carga llave1
00424BAC  .   8B7424 14     mov     esi,[esp+14]       // carga llave2
00424BB0  .   83C4 0C       add     esp,0C
00424BB3  .   8B08          mov     ecx,[eax]          // lee bytes [1..4] de la cadena
00424BB5  .   33CA          xor     ecx,edx            // los desencripta con llave1
00424BB7  .   8B50 04       mov     edx,[eax+4]        // lee bytes [5..8] de la cadena
00424BBA  .   8B40 08       mov     eax,[eax+8]        // lee bytes [9..12] de la cadena
00424BBD  .   33D6          xor     edx,esi            // desencripta bytes [5..8] con llave2
00424BBF  .   8B7424 0C     mov     esi,[esp+C]        // carga llave2
00424BC3  .   33C6          xor     eax,esi            // desencripta bytes [9..12] con llave2
00424BC5  .   3BCA          cmp     ecx,edx            // compara resultados
00424BC7  .   5E            pop     esi
00424BC8  .v  75 1C         jnz     short 00424BE6     // Salta a Salida2
00424BCA  .   3BD0          cmp     edx,eax            // vuelve a comparar
00424BCC  .v  75 18         jnz     short 00424BE6     // Salta a Salida2
00424BCE  .   81F9 20BF0200 cmp     ecx,2BF20          // y sigue..
00424BD4  .v  7C 10         jl      short 00424BE6     // Salta a Salida2
00424BD6  .   81F9 80EE3600 cmp     ecx,36EE80         // lo mismo..
00424BDC  .v  7F 08         jg      short 00424BE6     // Salta a Salida2
00424BDE  .   2BC2          sub     eax,edx
00424BE0  .   03C1          add     eax,ecx
00424BE2  .   83C4 0C       add     esp,0C
00424BE5  .   C3            ret                        // Salida1
00424BE6  >   B8 20BF0200   mov     eax,2BF20
00424BEB  .   83C4 0C       add     esp,0C
00424BEE  .   C3            ret                        // Salida2
```

En ella hace algunas operaciones tipo XOR con partes de la cadena a la que desviamos el puntero anterior "9mFUeAAAI1e" dejando un resultado en EAX que puede ser 0x2BF20 [Salida2] o bien otro [Salida1]. En cualquier caso este resultado lo salvará en otra variable al retornar de la rutina:

```
Address   Hex dump     Disassembly          Comment
00424D15  .  83C4 04   add    esp,4
00424D18  >  E8 63FEFFF call   00424B80
00424D1D  .  A3 80DE4600 mov   [46DE80],eax   // Salva el resultado
00424D22  .  33C0      xor    eax,eax
00424D24  .  C3        ret
```

Esta variable [46DE80] es la que perseguiremos, jeje.. si ponemos un BPM en dicha variable no para en ningún lado pero si buscamos todas las referencias a ésta con FIND REFERENCES - TO ADDRESS CONSTANT, aparecen varias pero la que me parece más interesante es ésta:

```
Address   Hex dump     Disassembly                    Comment
004251F0  $  A1 80DE4600 mov    eax,[46DE80]          // lee el flag
004251F5  .  8B5424 04   mov    edx,[esp+4]           // lee otro valor
004251F9  .  2BC2        sub    eax,edx               // los resta
004251FB  .  85C0        test   eax,eax
004251FD  .  A3 80DE4600 mov    [46DE80],eax          // salva el resultado
00425202  .v 7F 13       jg     short 00425217
00425204  .  E8 57FBFFFF call   00424D60              // interesante llamada xD
00425209  .  85C0        test   eax,eax
0042520B  .v 74 0A       je     short 00425217
0042520D  .  C705 80DE4600 mov  dword ptr [46DE80],36EE80  // hmmm..
00425217  >  C3          ret
```

En ella lee la variable, le resta un valor que vuelve a guardar en ella, lo comprueba y su es superior a cero salta hasta el RET evitando un call y poner en ella el valor constante 0x36EE80h, si entramos en el call no vemos nada interesante pero si volvemos a entrar vemos esto:

```
Address   Hex dump     Disassembly          Comment
00435C80  $  E8 9BF4FFFF call   00435120
00435C85  .  A0 A8064700 mov    al,[4706A8]
00435C8A  .  84C0        test   al,al
00435C8C  .v 74 03       je     short 00435C91   jmp to Reflexiv.radll_HasTheProductBeenPurchased
00435C8E  .  32C0        xor    al,al
00435C90  .  C3          ret
00435C91  >- FF25 64FF4600 jmp  [46FF64]         Reflexiv.radll_HasTheProductBeenPurchased
00435C97     90          nop
00435C98     90          nop
```

El nombre delata su función que es la de valorar si el programa fue registrado, si seguimos el "jmp [46FF64]" caemos en la librería ReflexiveArcade.dll:

```
Address   Hex dump     Disassembly          Comment
100080EF     55        push   ebp
100080F0     8BEC      mov    ebp,esp
100080F2     51        push   ecx
100080F3     53        push   ebx
100080F4     E8 58E4FFFF call  10006551
100080F9     84C0      test   al,al
100080FB  v  74 27     je     short 10008124
100080FD     68 1CB20C10 push  100CB21C            ASCII "radll_HasTheProductBeenPurchased"
10008102     8D4D FC   lea    ecx,[ebp-4]
10008105     E8 B9130100 call  100194C3
1000810A     51        push   ecx
1000810B     8D45 FC   lea    eax,[ebp-4]
1000810E     8BCC      mov    ecx,esp
10008110     50        push   eax
10008111     E8 99130100 call  100194AF
10008116     E8 CEE4FFFF call  100065E9
1000811B     59        pop    ecx
1000811C     8D4D FC   lea    ecx,[ebp-4]
1000811F     E8 6D140100 call  10019591
10008124     E8 04EEFFFF call  10006F2D            // Importante llamada!
10008129     8AD8      mov    bl,al
1000812B     E8 21E4FFFF call  10006551
10008130     84C0      test   al,al
10008132  v  74 27     je     short 1000815B
10008134     68 ECB10C10 push  100CB1EC            ASCII "radll_HasTheProductBeenPurchased
10008139     8D4D FC   lea    ecx,[ebp-4]
```

Dentro vemos este código, la llamada más importante está en :10008124 y debe retornar AL==1 para ir bien, dentro vemos:

```
Address   Hex dump     Disassembly          Comment
1000B729     55        push   ebp
1000B72A     8BEC      mov    ebp,esp
1000B72C     83EC 0C   sub    esp,0C
1000B72F     E8 8786FFFF call  10003DBB
1000B734     8378 38 FF cmp   dword ptr [eax+38],-1
1000B738  v  74 12     je     short 1000B74C
1000B73A     E8 7C86FFFF call  10003DBB
1000B73F     33C9      xor    ecx,ecx
1000B741     8378 38 01 cmp   dword ptr [eax+38],1
1000B745     0F94C1    sete   cl                    // cuidado aqui!
1000B748     8AC1      mov    al,cl
1000B74A     C9        leave
1000B74B     C3        ret
```

Esta es la primera de las dos salidas, realizamos un cambio:

```
Address   Hex dump     Disassembly                          Comment
1000B729  55           push   ebp
1000B72A  8BEC         mov    ebp,esp
1000B72C  83EC 0C      sub    esp,0C
1000B72F  E8 8786FFFF  call   10003DBB
1000B734  8378 38 FF   cmp    dword ptr [eax+38],-1
1000B738  74 12        je     short 1000B74C
1000B73A  E8 7C86FFFF  call   10003DBB
1000B73F  33C9         xor    ecx,ecx
1000B741  8378 38 01   cmp    dword ptr [eax+38],1
1000B745  B1 01        mov    cl,1                          // dejamos CL==1
1000B747  90           nop
1000B748  8AC1         mov    al,cl
1000B74A  C9           leave
1000B74B  C3           ret
```

Y ya tenemos una parte cubierta, la segunda está aquí:

```
Address   Hex dump     Disassembly                          Comment
1000B7A4  8958 38      mov    [eax+38],ebx
1000B7A7  8D4D F8      lea    ecx,[ebp-8]
1000B7AA  E8 E2DD0000  call   10019591
1000B7AF  8D4D F4      lea    ecx,[ebp-C]
1000B7B2  E8 DADD0000  call   10019591
1000B7B7  8AC3         mov    al,bl                         // Debe quedar AL==1
1000B7B9  5B           pop    ebx
1000B7BA  C9           leave
1000B7BB  C3           ret
```

Donde debemos tener AL==1 como indico en el comentario, la manera más sencilla es poner directamente este valor nosotros mismos:

```
Address   Hex dump     Disassembly                          Comment
1000B7A4  8958 38      mov    [eax+38],ebx
1000B7A7  8D4D F8      lea    ecx,[ebp-8]
1000B7AA  E8 E2DD0000  call   10019591
1000B7AF  8D4D F4      lea    ecx,[ebp-C]
1000B7B2  E8 DADD0000  call   10019591
1000B7B7  B0 01        mov    al,1                          // Debe quedar AL==1
1000B7B9  5B           pop    ebx
1000B7BA  C9           leave
1000B7BB  C3           ret
```

Estos dos cambios recordemos se aplican en "ReflexiveArcade.dll" seleccionamos ambos y picamos en COPY TO EXECUTABLE SELECTION - SAVE FILE para hacerlos permanentes en el disco, le dejamos el mismo nombre para probarlo, nos dejará al lado una copia de seguridad llamada "ReflexiveArcade.bak" que guardaremos por si las moscas. Probamos los cambios y...

Petardazo al canto, traceando descubrí el problema:

```
Address   Hex dump     Disassembly                          Comment
00435F85  A0 A8064700  mov    al,[4706A8]
00435F8A  84C0         test   al,al
00435F8C  75 0F        jnz    short 00435F9D
00435F8E  A0 A9064700  mov    al,[4706A9]
00435F93  84C0         test   al,al
00435F95  75 06        jnz    short 00435F9D
00435F97  FF25 8C064700 jmp   [47068C]                      // salto NULO ;(
00435F9D  32C0         xor    al,al
00435F9F  C3           ret
```

Se ejecuta un salto NULO, si hacemos un FOLLOW IN DUMP - MEMORY ADDRESS veremos que en la dirección [47068C] hay 0x00000000h que genera el error ya que es memoria protegida a la que no tenemos acceso. Si ejecutamos el EXE protegido también falla por lo que se deduce que no es un error de parcheo sino que detrás hay otro motivo. Renombro la librería a "ReflexiveArcade_org.dll" y la que tiene extensión .bak (la copia de seguridad) la dejo como "ReflexiveArcade.dll" ejecuto el dumpeado en Olly con un BPM ON ACCESS en la entrada [47068C] adonde apunta el salto, pulso F9 y Olly se detiene aquí:

```
Address   Hex dump        Disassembly                          Comment
004357AC  > 8B15 A006470( mov     edx,[4706A0]                 Reflexiv.10000000
004357B2  . 68 08484600   push    00464808                     ASCII "radll_HandleWindowsMessage"
004357B7  . 52            push    edx
004357B8  . FFD6          call    esi
004357BA  . 85C0          test    eax,eax
004357BC  . A3 8C064700   mov     [47068C],eax                 Reflexiv.radll_HandleWindowsMessage
004357C1  .v 75 24        jnz     short 004357E7
004357C3  . A1 A0064700   mov     eax,[4706A0]
004357C8  . 50            push    eax                          hLibModule => 10000000
004357C9  . FF15 9891450( call    [<&kernel32.FreeLibrary>]    FreeLibrary
004357CF  . 5F            pop     edi
```

Donde escribe en ella la dirección de una función de dicha DLL, por algún motivo con librería la modificada aquí no para, el problema está justo encima:

```
Address   Hex dump        Disassembly                          Comment
0043551B  > 8D4C24 0C     lea     ecx,[esp+C]
0043551F  . 51            push    ecx
00435520  . E8 8BF9FFFF   call    00434EB0                     // call verificador
00435525  . 83C4 04       add     esp,4
00435528  . 84C0          test    al,al
0043552A  .v 75 25        jnz     short 00435551
0043552C  . 8B15 A006470( mov     edx,[4706A0]                 Reflexiv.10000000
00435532  . 52            push    edx                          hLibModule => 10000000
00435533  . FF15 9891450( call    [<&kernel32.FreeLibrary>]    FreeLibrary
00435539  . 5F            pop     edi
0043553A  . 5E            pop     esi
0043553B  . C705 9406470( mov     dword ptr [470694],00464C    ASCII "The ReflexiveArcade.dll h
00435545  . 66:B8 0500    mov     ax,5
00435549  . 5B            pop     ebx
0043554A  . 81C4 0401000( add     esp,104
00435550  . C3            ret
00435551  > A1 A0064700   mov     eax,[4706A0]
00435556  . 8B35 A892450( mov     esi,[<&kernel32.GetProcAd    kernel32.GetProcAddress
0043555C  . 68 FC4B4600   push    00464BFC                     ProcNameOrOrdinal = "radll_GetDL
00435561  . 50            push    eax                          hModule => 10000000 (Reflexiv)
00435562  . FFD6          call    esi                          GetProcAddress
00435564  . 85C0          test    eax,eax
00435566  . A3 88064700   mov     [470688],eax
0043556B  .v 75 25        jnz     short 00435592
0043556D  . 8B0D A006470( mov     ecx,[4706A0]                 Reflexiv.10000000
```

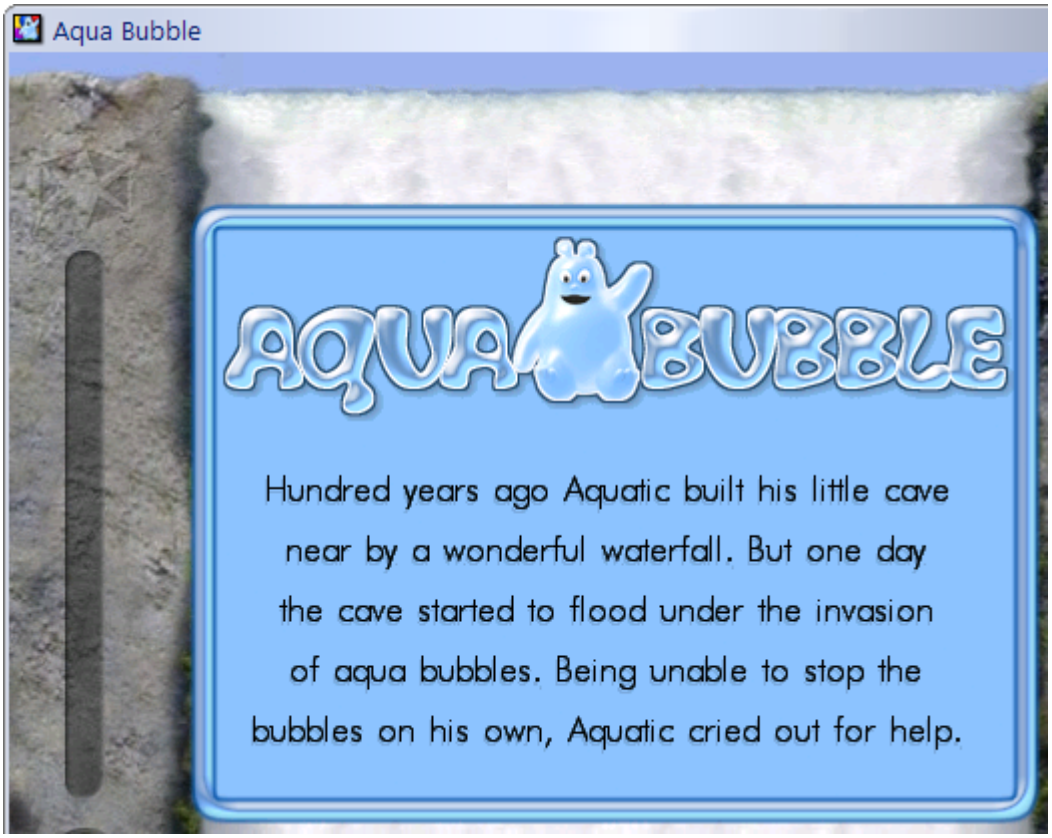
Ese JNZ se produce en la DLL original y no en la modificada, es muy probable que dentro de éste se haga alguna verificación y en caso de no encontrar los resultados esperados retorna AL==0, veamos la llamada por dentro:

```
Address   Hex dump        Disassembly                          Comment
0043509E  . 6A 00         push    0
004350A0  . 68 C0444600   push    004644C0                     ASCII "ReflexiveArcade DLL Signature"
004350A5  . 25 FFFF0000   and     eax,0FFFF
004350AA  . 52            push    edx
004350AB  . 8B5424 20     mov     edx,[esp+20]
004350AF  . 8D8C24 2C040( lea     ecx,[esp+42C]
004350B6  . 50            push    eax
004350B7  . 51            push    ecx
004350B8  . 52            push    edx
004350B9  . FF15 1890450( call    [<&advapi32.CryptVerifySi    ADVAPI32.CryptVerifySignatureA
004350BF  . 85C0          test    eax,eax
004350C1  . 8B4424 18     mov     eax,[esp+18]
004350C5  . 50            push    eax
004350C6  .v 75 2B        jnz     short 004350F3
004350C8  . FF15 1490450( call    [<&advapi32.CryptDestroyK    ADVAPI32.CryptDestroyKey
004350CE  . 8B4C24 14     mov     ecx,[esp+14]
004350D2  . 51            push    ecx
004350D3  . FF15 1C90450( call    [<&advapi32.CryptDestroyH    ADVAPI32.CryptDestroyHash
004350D9  . 8B5424 10     mov     edx,[esp+10]
004350DD  . 6A 00         push    0
004350DF  . 52            push    edx
004350E0  . FF15 2890450( call    [<&advapi32.CryptReleaseC    ADVAPI32.CryptReleaseContext
004350E6  . 5F            pop     edi
004350E7  . 5E            pop     esi
004350E8  . 5D            pop     ebp
004350E9  . 32C0          xor     al,al
004350EB  . 5B            pop     ebx
004350EC  . 81C4 1008000( add     esp,810
004350F2  . C3            ret
004350F3  > FF15 1490450( call    [<&advapi32.CryptDestroyK    ADVAPI32.CryptDestroyKey
004350F9  . 8B4C24 14     mov     ecx,[esp+14]
```

Vaya, vaya, hmmm.., está claro lo que hace ese "call CryptVerify.." de la librería Advapi32.dll, el propio nombre la delata, debemos JMPear el salto [JNZ] de manera que salte siempre:

```
Address   Hex dump     Disassembly                          Comment
004350B9  .  FF15 1890450( call   [<&advapi32.CryptVerifySi  ADVAPI32.CryptVerifySignatureA
004350BF  .  85C0        test   eax,eax
004350C1  .  8B4424 18   mov    eax,[esp+18]
004350C5  .  50          push   eax
004350C6     EB 2B       jmp    short 004350F3
004350C8  .  FF15 1490450( call   [<&advapi32.CryptDestroyK  ADVAPI32.CryptDestroyKey
004350CE  .  8B4C24 14   mov    ecx,[esp+14]
004350D2  .  51          push   ecx
004350D3  .  FF15 1C90450( call   [<&advapi32.CryptDestroyH  ADVAPI32.CryptDestroyHash
004350D9  .  8B5424 10   mov    edx,[esp+10]
```

Salvamos el cambio [ahora en tute_.exe] y le decimos que sobrescriba el anterior EXE con este nombre para que lo actualice sin crear ningún otro, volvemos a colocarle la DLL modificada y lo probamos..



El programa al fin corre sin ninguna limitación, jeje.. al final no ha sido tan duro como parecía, vemos la importancia una vez más de las variables [flags] que muchas veces nos llevan a la zona caliente, es cuestión de aprovechar el filón.

Para terminar eliminamos la sección .adata que no necesitamos para nada, podemos eliminar también la sección .data pero por alguna razón pierde el icono por lo que la dejamos por si las moscas, esto lo hacemos con PEditor v1.7 y luego arrastramos el EXE sobre el LordPE para que le haga un Rebuild y ya lo tenemos, en este caso no ha disminuido excesivamente su tamaño [de 616Kb a 611Kb] pero en algunos casos sí se reduce considerablemente por lo que merece la pena hacerlo.

Y aquí termina el tute de hoy, espero escribir más y sobre todo mejores en lo sucesivo, para cualquier cosa ya sabéis donde encontrarme: syxe05@gmail.com

Dedicado a TODA la lista de CracksLatinoS SIN EXCEPCIÓN¡¡!!!